10

Arrays and Addressing Modes

Overview

In some applications, it is necessary to treat a collection of values as a group. For example, we might need to read a set of test scores and print the median score. To do so, we would first have to store the scores in ascending order (this could be done as the scores are entered, or they could be sorted after they are all in memory). The advantage of using an array to store the data is that a single name can be given to the whole structure, and an element can be accessed by providing an index.

In section 10.1 we show how one-dimensional arrays are declared in assembly language. To access the elements, in section 10.2 we introduce new ways of expressing operands—the register indirect, based and indexed addressing modes. In section 10.3, we use these addressing modes to sort an array.

A two-dimensional array is a one-dimensional array whose elements are also one-dimensional arrays (an array of arrays). In section 10.4, we show how they are stored. These arrays have two indexes, and are most easily manipulated by the based indexed addressing mode of section 10.5. Section 10.6 provides a simple application.

Section 10.7 introduces the XLAT (translate) instruction. This instruction is useful when we want to do data conversion; we use it to encode and decode a secret message.

10.1 One-Dimensional Arrays

A **one-dimensional array** is an ordered list of elements, all of the same type. By "ordered," we mean that there is a first element, second element, third element, and so on. In mathematics, if A is an array, the elements

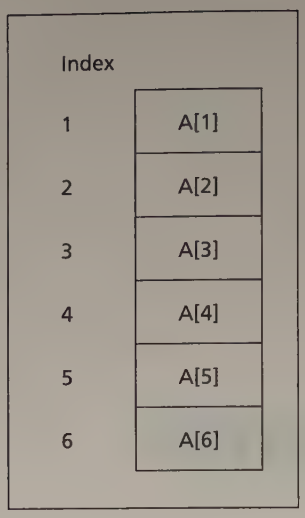

Figure 10.1 A One-Dimensional Array A

are usually denoted by A[1], A[2], A[3], and so on. Figure 10.1 shows a one-dimensional array A with six elements.

In Chapter 4, we used the DB and DW pseudo-ops to declare byte and word arrays; for example, a five-character string named MSG,

```
MSG  DB    'abcde'
```

or a word array W of six integers, initialized to 10,20,30,40,50,60.

```
W   DW      10,20,30,40,50,60
```

The address of the array variable is called the **base address of the array.** If the offset address assigned to W is 0200h, the array looks like this in memory:

Offset address	*Symbolic address*	*Decimal content*
0200h	W	10
0202h	W+2h	20
0204h	W+4h	30
0206h	W+6h	40
0208h	W+8h	50
020Ah	W+Ah	60

The DUP Operator

It is possible to define arrays whose elements share a common initial value by using the **DUP** (duplicate) operator. It has this form:

```
repeat_count        DUP (value)
```

This operator causes value to be repeated the number of times specified by repeat_count. For example,

```
GAMMA  DW 100 DUP (0)
```

sets up an array of 100 words, with each entry initialized to 0. Similarly,

```
DELTA  DB 212 DUP (?)
```

creates an array of 212 uninitialized bytes. DUPs may be nested. For example,

```
LINE  DB      5,4, 3 DUP (2, 3 DUP (0), 1)
```

which is equivalent to

```
LINE  DB      5,4,2,0,0,0,1,2,0,0,0,1,2,0,0,0,1
```

Location of Array Elements

The address of an array element may be specified by adding a constant to the base address. Suppose A is an array and S denotes the number of bytes in an element (S = 1 for a byte array, S = 2 for a word array). The position of the elements in array A can be determined as follows:

Position	*Location*
1	A
2	A = 1 × S
3	A = 2 × S
.	.
.	.
.	.
N	A = (*N* – 1) × S

Example 10.1 Exchange the 10th and 25th elements in a word array W.

Solution: W[10] is located at address W + 9 × 2 = W + 18 and W[25] is at W + 24 × 2 = W + 48, so we can do the exchange as follows:

```
MOV  AX,W+18        ;AX has W[10]
XCHG W+48,AX        ;AX has W[25]
MOV  W+18,AX        ;complete exchange
```

In many applications, we need to perform some operation on each element of an array. For example, suppose array A is a 10-element array, and we want to add the elements. In a high-level language, we could do it like this:

```
sum = 0
N = 1
REPEAT
   sum = sum + A[N]
   N = N + 1
UNTIL N > 10
```

To code this in assembly language, we need a way to move from one array element to the next one. In the next section, we'll see how to accomplish this by indirect addressing.

10.2 Addressing Modes

The way an operand is specified is known as its **addressing mode**. The addressing modes we have used so far are (1) **register mode**, which means that an operand is a register; (2) **immediate mode**, when an operand is a constant; and (3) **direct mode**, when an operand is a variable. For example,

```
MOV AX,0          (Destination AX is register mode,
                   source 0 is immediate mode.)
ADD ALPHA,AX      (Destination ALPHA is direct mode,
                   source AX is register mode.)
```

There are four additional addressing modes for the 8086: (1) Register Indirect, (2) Based, (3) Indexed, and (4) Based Indexed. These modes are used to address memory operands indirectly. In this section, we discuss the first three of these modes; they are useful in one-dimensional array processing. Based indexed mode can be used with two-dimensional arrays; it is covered in section 10.5.

10.2.1 Register Indirect Mode

In this mode, the offset address of the operand is contained in a register. We say that the register acts as a **pointer** to the memory location. The operand format is

```
[register]
```

The register is BX, SI, DI, or BP. For BX, SI, or DI, the operand's segment number is contained in DS. For BP, SS has the segment number.

For example, suppose that SI contains 0100h, and the word at 0100h contains 1234h. To execute

```
MOV   AX,[SI]
```

the CPU (1) examines SI and obtains the offset address 100h, (2) uses the address DS:0100h to obtain the value 1234h, and (3) moves 1234h to AX. This is not the same as

```
MOV   AX,SI
```

which simply moves the value of SI, namely 100h, into AX.

Example 10.2 Suppose that

BX contains 1000h	Offset 1000h contains 1BACh
SI contains 2000h	Offset 2000h contains 20FEh
DI contains 3000h	Offset 3000h contains 031Dh

where the above offsets are in the data segment addressed by DS.

Tell which of the following instructions are legal. If legal, give the source offset address and the result or number moved.

a. `MOV BX,[BX]`
b. `MOV CX,[SI]`
c. `MOV BX,[AX]`
d. `ADD [SI],[DI]`
e. `INC [DI]`

Solution:

	Source offset	*Result*
a.	1000h	1BACh
b.	2000h	20FEh
c.	illegal source register	(must be BX, SI, or DI)
d.	illegal memory–memory addition	
e.	3000h	031Eh

Now let's return to the problem of adding the elements of an array.

Example 10.3 Write some code to sum in AX the elements of the 10-element array W defined by

```
W  DW    10,20,30,40,50,60,70,80,90,100
```

Solution: The idea is to set a pointer to the base of the array, and let it move up the array, summing elements as it goes.

```
      XOR  AX,AX          ;AX holds sum
      LEA  SI,W           ;SI points to array W
      MOV  CX,10          ;CX has number of elements
ADDNOS:
      ADD  AX,[SI]        ;sum = sum + element
      ADD  SI,2           ;move pointer to the next
                          ;element
      LOOP ADDNOS         ;loop until done
```

Here we must add 2 to SI on each trip through the loop because W is a word array (recall from Chapter 4 that LEA moves the source offset address into the destination).

The next example shows how register indirect mode can be used in array processing.

Example 10.4 Write a procedure REVERSE that will reverse an array of *N* words. This means that the *N*th word becomes the first, the (*N*–1)st word becomes the second, and so on, and the first word becomes the *N*th word. The procedure is entered with SI pointing to the array, and BX has the number of words *N*.

Solution: The idea is to exchange the 1st and *N*th words, the 2nd and (*N*–1)st words, and so on. The number of exchanges will be *N*/2 (rounded down to the nearest integer if *N* is odd). Recall from section 10.1 that the *N*th element in a word array A has address $A + 2 \times (N - 1)$.

Program Listing PGM10_1.ASM

```
REVERSE PROC
;reverses a word array
;input: SI = offset of array
;       BX = number of elements
;output: reversed array
```

```
        PUSH   AX           ;save registers
        PUSH   BX
        PUSH   CX
        PUSH   SI
        PUSH   DI
;make DI point to nth word
        MOV    DI,SI        ;DI pts to 1st word
        MOV    CX,BX        ;CX = n
        DEC    BX           ;BX = n-1
        SHL    BX,1         ;BX = 2 x (n-1)
        ADD    DI,BX        ;DX pts to nth word
        SHR    CX,1         ;CX = n/2 = no. of swaps to do
;swap elements
XCHG_LOOP:
        MOV    AX,[SI]      ;get an elt in lower half of array
        XCHG   AX,[DI]      ;insert in upper half
        MOV    [SI],AX      ;complete exchange
        ADD    SI,2         ;move ptr
        SUB    DI,2         ;move ptr
        LOOP   XCHG_LOOP    ;loop until done
        POP    DI           ;restore registers
        POP    SI
        POP    CX
        POP    BX
        POP    AX
        RET
REVERSE ENDP
```

10.2.2 Based and Indexed Addressing Modes

In these modes, the operand's offset address is obtained by adding a number called a **displacement** to the contents of a register. Displacement may be any of the following:

the offset address of a variable

a constant (positive or negative)

the offset address of a variable plus or minus a constant

If A is a variable, examples of displacements are:

A (offset address of a variable)

–2 (constant)

A + 4 (offset address of a variable plus a constant)

The syntax of an operand is any of the following equivalent expressions:

```
[register + displacement]
[displacement + register]
[register] + displacement
displacement + [register]
displacement[register]
```

The register must be BX, BP, SI, or DI. If BX, SI, or DI is used, DS contains the segment number of the operand's address. If BP is used, SS has the segment number. The addressing mode is called **based** if BX (base register) or

BP (base pointer) is used; it is called **indexed** if SI (source index) or DI (destination index) is used.

For example, suppose W is a word array, and BX contains 4. In the instruction

```
MOV AX,W[BX]
```

the displacement is the offset address of variable W. The instruction moves the element at address W + 4 to AX. This is the third element in the array. The instruction could also have been written in any of these forms:

```
MOV AX,[W+BX]
MOV AX,[BX+W]
MOV AX,W+[BX]
MOV AX,[BX]+W
```

As another example, suppose SI contains the address of a word array W. In the instruction

```
MOV AX,[SI+2]
```

the displacement is 2. The instruction moves the contents of W + 2 to AX. This is the second element in the array. The instruction could also have been written in any of these forms:

```
MOV AX,[2+SI]
MOV AX,2+[SI]
MOV AX,[SI]+2
MOV AX,2[SI]
```

Example 10.5 Rework example 10.3 by using based mode.

Solution: The idea is to clear base register BX, then add 2 to it on each trip through the summing loop.

```
      XOR  AX,AX          ;AX holds sum
      XOR  BX,BX          ;clear base register
      MOV  CX,10          ;CX has number of elements
ADDNOS:
      ADD  AX,W[BX]       ;sum = sum + element
      ADD  BX,2           ;index next element
      LOOP ADDNOS         ;loop until done
```

Example 10.6 Suppose that ALPHA is declared as

```
ALPHA  DW  0123h,0456h,0789h,0ABCDh
```

in the segment addressed by DS. Suppose also that

BX contains 2
SI contains 4
DI contains 1

Offset 0002 contains 1084h
Offset 0004 contains 2BACh

Tell which of the following instructions are legal. If legal, give the source offset address and the number moved.

a. `MOV AX,[ALPHA+BX]`
b. `MOV BX,[BX+2]`
c. `MOV CX,ALPHA[SI]`
d. `MOV AX,-2[SI]`
e. `MOV BX,[ALPHA+3+DI]`
f. `MOV AX,[BX]2`
g. `ADD BX,[ALPHA+AX]`

Solution:

	Source offset	*Number moved*
a.	ALPHA+2	0456h
b.	2+2 = 4	2BACh
c.	ALPHA+4	0789h
d.	–2+4 = 2	1084h
e.	ALPHA+3+1 = ALPHA+4	0789h
f.	Illegal form of source operand	
g.	Illegal source register	

The next two examples illustrate array processing by based and indexed modes.

Example 10.7 Replace each lowercase letter in the following string by its upper case equivalent. Use index addressing mode.

```
MSG DB          'this is a message'
```

Solution:

```
      MOV  CX,17          ;no. of chars in string
      XOR  SI,SI          ;SI indexes a char
TOP:
      CMP  MSG[SI],' '    ;blank?
      JE   NEXT           ;yes, skip over
      AND  MSG[SI],0DFh   ;no, convert to upper case
NEXT:
      INC  SI             ;index next byte
      LOOP TOP            ;loop until done
```

10.2.3 *The PTR Operator and the LABEL Pseudo-op*

You saw in Chapter 4 that the operands of an instruction must be of the same type; for example, both bytes or both words. If one operand is a constant, the assembler attempts to infer the type from the other operand. For example, the assembler treats the instruction

```
MOV AX,1
```

as a word instruction, because AX is a 16-bit register. Similarly, it treats

```
MOV BH,5
```

as a byte instruction. However, it can't assemble

```
MOV [BX],1          ;illegal
```

because it can't tell whether the destination is the byte pointed to by BX or the word pointed to by BX. If you want the destination to be a byte, you can say,

```
MOV BYTE PTR [BX],1
```

and if you want the destination to be a word, you say,

```
MOV WORD PTR [BX],1
```

Example 10.8 In the string of example 10.7, replace the character "t" by "T".

Solution 1: Using register indirect mode,

```
LEA SI,MSG              ;SI points to MSG
MOV BYTE PTR [SI],'T'    ;replace 't' by 'T'
```

Solution 2: Using index mode

```
XOR SI,SI              ;clear SI
MOV MSG[SI],'T'        ;replace 't' by 'T'
```

Here it is not necessary to use the PTR operator, because MSG is a byte variable.

Using PTR to Override a Type

In general, the PTR operator can be used to override the declared type of an address expression. The syntax is

```
type PTR address_expression
```

where the type is BYTE, WORD, or DWORD (doubleword), and the address expression has been typed as DB, DW, or DD.

For example, suppose you have the following declaration:

```
DOLLARS  DB   1Ah
CENTS    DB   52h
```

and you'd like to move the contents of DOLLARS to AL and CENTS to AH with a single MOV instruction. Now

```
MOV AX,DOLLARS    ;illegal
```

is illegal because the destination is a word and the source has been typed as a byte variable. But you can override the type declaration with WORD PTR as

```
MOV AX, WORD PTR DOLLARS  ;AL = dollars, AH = cents
```

and the instruction will move 521Ah to AX.

The LABEL Pseudo-Op

Actually, there is another way to get around the problem of type conflict in the preceding example. Using the **LABEL** pseudo-op, we could declare

```
MONEY          LABEL          WORD
DOLLARS        DB             1Ah
CENTS          DB             52h
```

This declaration types MONEY as a word variable, and the components DOLLARS and CENTS as byte variables, with MONEY and DOLLARS being assigned the same address by the assembler. The instruction

```
MOV AX, MONEY          ;AL = dollars, AH = cents
```

is now legal. So are the following instructions, which have the same effect:

```
MOV AL, DOLLARS
MOV AH, CENTS
```

Example 10.9 Suppose the following data are declared:

```
.DATA
A DW  1234h
B LABEL BYTE
  DW 5678h
C LABEL WORD
C1 DB 9Ah
C2 DB 0BCh
```

Tell whether the following instructions are legal, and if so, give the number moved.

	Instruction
a.	`MOV AX,B`
b.	`MOV AH,B`
c.	`MOV CX,C`
d.	`MOV BX, WORD PTR B`
e.	`MOV DL, BYTE PTR C`
f.	`MOV AX, WORD PTR C1`

Solution:

a. illegal—type conflict
b. legal, 78h
c. legal, 0BC9Ah
d. legal, 5678h
e. legal, 9Ah
f. legal, 0BC9Ah

10.2.4 *Segment Override*

In register indirect mode, the pointer register BX, SI, or DI specifies an offset address relative to DS. It is also possible to specify an offset relative to one of the other segment registers. The form of an operand is

```
segment_register:[pointer_register]
```

For example,

```
MOV AX,ES:[SI]
```

If SI contains 0100h, the source address in this instruction is ES:0100h. You might want to do this in a program with two data segments, where ES contains the segment number of the second data segment.

Segment overrides can also be used with based and indexed modes.

10.2.5 Accessing the Stack

We mentioned earlier that when BP specifies an offset in register indirect mode, SS supplies the segment number. This means that BP may be used to access items on the stack.

Example 10.10 Move the top three words on the stack into AX, BX, and CX without changing the stack.

Solution:

```
MOV BP,SP          ;BP points to stacktop
MOV AX,[BP]        ;move stacktop to AX
MOV BX,[BP+2]      ;move second word to BX
MOV CX,[BP+4]      ;move third word to CX
```

A primary use of BP is to pass values to a procedure (see Chapter 14).

10.3 An Application: Sorting an Array

It is much easier to locate an item in an array if the array has been sorted. There are dozens of sorting methods; the method we will discuss here is called *selectsort*. It is one of the simplest sorting methods.

To sort an array A of N elements, we proceed as follows:

Pass 1. Find the largest element among A[1] . . . A[N]. Swap it and A[N]. Because this puts the largest element in position N, we need only sort A[1] . . . A[N–1] to finish.

Pass 2. Find the largest element among A[1] . . . A[N–1]. Swap it and A[N–1]. This places the next-to-largest element in its proper position.

.

.

.

Pass N–1. Find the largest element among A[1], A[2]. Swap it and A[2]. At this point A[2] . . . A[N] are in their proper positions, so A[1] is as well, and the array is sorted.

For example, suppose the array A consists of the following integers:

Position	*1*	*2*	*3*	*4*	*5*
initial data	21	5	16	40	7
pass 1	21	5	16	7	40
pass 2	7	5	16	21	40
pass 3	7	5	16	21	40
pass 4	5	7	16	21	40

Selectsort algorithm

```
i = N
FOR N-1 times DO
  Find the position k of the largest element
     among A[1]..A[i]
(*) Swap A[k] and A[i]
  i = i-1
END_FOR
```

Step (*) will be handled by a procedure SWAP. The code for the procedures is the following (we'll suppose the array to be sorted is a byte array):

Program Listing PGM10_2.ASM

```
1:  SELECT  PROC
2:  ;sorts a byte array by the selectsort method
3:  ;input:  SI = array offset address
4:  ;          BX = number of elements
5:  ;output: SI = offset of sorted array
6:  ;uses: SWAP
7:          PUSH  BX
8:          PUSH  CX
9:          PUSH  DX
10:         PUSH  SI
11:         DEC   BX          ;N = N-1
12:         JE    END_SORT    ;exit if 1-elt array
13:         MOV   DX,SI       ;save array offset
14: ;for N-1 times do
15: SORT_LOOP:
16:         MOV   SI,DX       ;SI pts to array
17:         MOV   CX,BX       ;no. of comparisons to make
18:         MOV   DI,SI       ;DI pts to largest element
19:         MOV   AL,[DI]     ;AL has largest element
20: ;locate biggest of remaining elts
21: FIND_BIG:
22:         INC   SI          ;SI pts to next element
23:         CMP   [SI],AL     ;is new element > largest?
24:         JNG   NEXT        ;no, go on
25:         MOV   DI,SI       ;yes, move DI
26:         MOV   AL,[DI]     ;AL has largest element
27: NEXT:
28:         LOOP  FIND_BIG    ;loop until done
29: ;swap biggest elt with last elt
30:         CALL  SWAP        ;Swap with last elt
31:         DEC   BX          ;N = N-1
32:         JNE   SORT_LOOP   ;repeat if N <> 0
33: END_SORT:
34:         POP   SI
35:         POP   DX
36:         POP   CX
37:         POP   BX
38:         RET
39: SELECT  ENDP
40: SWAP    PROC
41: ;swaps two array elements
42: ;input: SI = one element
43: ;         DI = other element
```

```
44: ;output: exchange elements
45:          PUSH   AX           ;save AX
46:          MOV    AL,[SI]      ;get A[i]
47:          XCHG   AL,[DI]      ;place in A[k]
48:          MOV    [SI],AL      ;put A[k] in A[i]
49:          POP    AX           ;restore AX
50:          RET
51: SWAP     ENDP
```

Procedure SELECT is entered with the array offset address in SI, and the number of elements N in BX. The algorithm sorts the array in $N - 1$ passes so BX is decremented; if it contains 0, then we have a one-element array and there is nothing to do, so the procedure exits.

In the general case, the procedure enters as the main processing loop (lines 15–32). Each pass through this loop places the largest of the remaining unsorted elements in its proper place.

In lines 21–28, a loop is entered to find the largest of the remaining unsorted elements; the loop is exited with DI pointing to the largest element and SI pointing to the last element in the array. At line 30, procedure SWAP is called to exchange the elements pointed to by SI and DI.

The procedure can be tested by inserting them in a testing program.

Program Listing PGM10_3.ASM

```
TITLE PGM10_3: TEST SELECT
.MODEL SMALL
.STACK 100H
.DATA
A      DB     5,2,1,3,4
.CODE
MAIN   PROC
       MOV    AX,@DATA
       MOV    DS,AX
       LEA    SI,A
       MOV    BX,5
       CALL   SELECT
       MOV    AH,4CH
       INT    21H
MAIN   ENDP
;select goes here
       END  MAIN
```

After assembling and linking, we enter DEBUG and execute down to the procedure call (the addresses in the following demonstration were determined in a previous DEBUG session):

```
-GC
AX=100D BX=0005 CX=0049 DX=0000 SP=0100 BP=0000 SI=0004 DI=0000
DS=100D ES=0FF9 SS=100E CS=1009 IP=000C NV UP EI PL NZ NA PO NC
1009:000C E80400    CALL  0013
```

Before calling the procedure, let's look at the unsorted array:

```
-D4 8
100D:0000     05 02 01 03-04
```

The data appear in the order 5, 2, 1, 3, 4. Now let's execute SELECT:

```
-GF
AX=1002 BX=0005 CX=0049 DX=0000 SP=0100 BP=0000 SI=0004 DI=0005
DS=100D ES=0FF9 SS=100E CS=1009 IP=000F NV UP EI PL ZR NA PE NC
1009:000F B44C    MOV  AH,4C
```

and look at the array again:

```
-D4 8
100D:0000    01 02 03 04-05
```

It is now in ascending order.

10.4 Two-Dimensional Arrays

A **two-dimensional array** is an array of arrays; that is, a one-dimensional array whose elements are one-dimensional arrays. We can picture the elements as being arranged in rows and columns. Figure 10.2 shows a two-dimensional array B with three rows and four columns (a 3×4 array); B[i,j] is the element in row i and column j.

How Two-Dimensional Arrays Are Stored

Because memory is one-dimensional, the elements of a two-dimensional array must be stored sequentially. There are two commonly used ways: In **row-major order**, the row 1 elements are stored, followed by the row 2 elements, then the row 3 elements, and so on. In **column-major order**, the elements of the first column are stored, followed by the second column, third column, and so on. For example, suppose array B has 10, 20, 30, and 40 in the first row, 50, 60, 70, and 80 in the second row, and 90, 100, 110, and 120 in the third row. It could be stored in row-major order as follows:

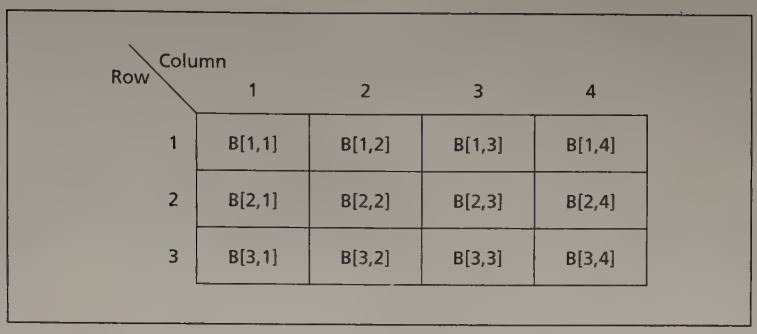

Figure 10.2 A Two-Dimensional Array B

```
B   DW   10,20,30,40
    DW   50,60,70,80
    DW   90,100,110,120
```

or in column-major order as follows:

```
B   DW   10,50,90
    DW   20,60,100
    DW   30,70,110
    DW   40,80,120
```

Most high-level language compilers store two-dimensional arrays in row-major order. In assembly language, we can do it either way. If the elements of a row are to be processed together sequentially, then row-major order is better, because the next element in a row is the next memory location. Conversely, column-major order is better if the elements of a column are to be processed together.

Locating an Element in a Two-Dimensional Array

Suppose an $M \times N$ array A is stored in row-major order, where the size of the elements is S ($S = 1$ for a byte array, $S = 2$ for a word array). To find the location of A[i, j],

1. Find where row i begins.
2. Find the location of the jth element in that row.

Here is the first step. Row 1 begins at location A. Because there are N elements in each row, each of size S bytes, Row 2 begins at location A + $N \times S$, Row 3 begins at location A + $2 \times N \times S$, and in general, Row i begins at location A + $(i - 1) \times N \times S$.

Now for the second step. We know from our discussion of one-dimensional arrays that the jth element in a row is stored $(j - 1) \times S$ bytes from the beginning of the row.

Adding the results of steps 1 and 2, we get the final result:

If A is an $M \times N$ array, with element size S bytes, stored in row-major order, then

(1) A[i, j] has address A+$((i - 1) \times N + (j - 1)) \times S$

There is a similar expression for column-major ordered arrays:

If A is an $M \times N$ array, with element size S, stored in column-major order, then

(2) A[i,j] has address A + $((i - 1) + (j - 1) \times M) \times S$

Example 10.12 Suppose A is an $M \times N$ word array stored in row-major order.

1. Where does row i begin?
2. Where does column j begin?
3. How many bytes are there between elements in a column?

Solution:

1. Row i begins at A[i, 1]; by formula (1) its address is $A + (i - 1) \times N \times 2$.
2. Column j begins at A[1, j]; by formula (1) the address is $A + (j - 1) \times 2$.
3. Because there are N columns, there are $2 \times N$ bytes between elements in any given column.

10.5 Based Indexed Addressing Mode

In this mode, the offset address of the operand is the sum of

1. the contents of a base register (BX or BP)
2. the contents of an index register (SI or DI)
3. optionally, a variable's offset address
4. optionally, a constant (positive or negative)

If BX is used, DS contains the segment number of the operand's address; if BP is used, SS has the segment number. The operand may be written several ways; four of them are

1. `variable[base_register][index_register]`
2. `[base_register + index_register + variable + constant]`
3. `variable[base_register + index_register + constant]`
4. `constant[base_register + index_register + variable]`

The order of terms within these brackets is arbitrary.

For example, suppose W is a word variable, BX contains 2, and SI contains 4. The instruction

```
MOV AX,W[BX][SI]
```

moves the contents of W+2+4 = W+6 to AX. This instruction could also have been written in either of these ways:

```
MOV AX,[W+BX+SI]
```

or

```
MOV AX,W[BX+SI]
```

Based indexed mode is especially useful for processing two-dimensional arrays, as the following example shows.

Example 10.13 Suppose A is a 5- × 7-word array stored in row-major order. Write some code to (1) clear row 3, (2) clear column 4. Use based indexed mode.

Solution:

1. From example 10.12, we know that in an *M*- × *N*-word array A, row i begins at A + (i – 1) × *N* × 2. Thus in a 5 × 7 array, row 3 begins at A + (3 – 1) × 7 × 2 = A + 28. So we can clear row 3 as follows:

```
      MOV X,28;          BX indexes row 3
      XOR SI,SI          ;SI will index columns
      MOV CX,7           ;number of elements in a row
CLEAR:
      MOV A[BX][SI],0    ;clear A[3,j]
      ADD SI,2           ;go to next column
      LOOP CLEAR         ;loop until done
```

2. Again from example 10.12, column j begins at A + (j – 1) × 2 in an *M*- × *N*-word array. Thus column 4 begins at A + (4 – 1) × 2 = A + 6. Since A is a seven-column word array stored in row-major order, to get to the next element in column 4 we need to add 7 × 2 = 14. We can clear column 4 as follows:

```
      MOV SI,6           ;SI will index column 4
      XOR BX,BX          ;BX will index rows
      MOV CX,5           ;number of elements in a column
CLEAR:
      MOV A[BX][SI],0    ;clear A[i,4]
      ADD BX,1           ;go to next row
      LOOP CLEAR         ;loop until done
```

10.6 An Application: Averaging Test Scores

Suppose a class of five students is given four exams. The results are recorded as follows:

	Test 1	*Test 2*	*Test 3*	*Test 4*
MARY ALLEN	67	45	98	33
SCOTT BAYLIS	70	56	87	44
GEORGE FRANK	82	72	89	40
BETH HARRIS	80	67	95	50
SAM WONG	78	76	92	60

We will write a program to find the class average on each exam. To do this, we sum the entries in each column and divide by 5.

Algorithm

```
1. j = 4
2. REPEAT
3.   sum the scores in column j
4.   divide sum by 5 to get the average in column j
4.   j = j-1
5. UNTIL j = 0
```

We choose to start summing in column 4 because it makes the code a little shorter. Step 3 may be broken down further as follows:

```
sum[j] = 0
i = 1
FOR 5 times DO
  sum[j] = sum[j] + score[i,j]
  i = 1+1
END_FOR
```

Program Listing PGM10_4.ASM

```
0:  TITLE PGM10_4: CLASS AVERAGE
1:  .MODEL   SMALL
2:  .STACK   100H
3:  .DATA
4:  FIVE     DW     5
5:  SCORES   DW     67,45,98,33  ;Mary Allen
6:           DW     70,56,87,44  ;Scott Baylis
7:           DW     82,72,89,40  ;George Frank
8:           DW     80,67,95,50  ;Beth Harris
9:           DW     78,76,92,60  ;Sam Wong
10: AVG      DW     5 DUP (0)
11: .CODE
12: MAIN     PROC
13:          MOV    AX,@DATA
14:          MOV    DS,AX        ;initialize DS
15: ;j=4
16:          MOV    SI,6         ;col index, initially col 4
17: REPEAT:
18:          MOV    CX,5         ;no. of rows
19:          XOR    BX,BX        ;row index, initially 1
20:          XOR    AX,AX        ;col_sum, initially 0
21: ;sum scores in column j
22: FOR:
23:          ADD    AX,SCORES[BX+SI];col_sum=col_sum + score
24:          ADD    BX,8         ;index next row
25:          LOOP   FOR          ;keep adding scores
26: ;endfor
27: ;compute average in column j
28:          XOR    DX,DX        ;clear high part of divnd
29:          DIV    FIVE         ;AX =  average
30:          MOV    AVG[SI],AX   ;store in array
31:          SUB    SI,2         ;go to next column
32:          ;until j=0
33:          JNL    REPEAT       ;unless SI < 0
34: ;dos exit
35:          MOV    AH,4CH
36:          INT    21H
37: MAIN     ENDP
38:          END  MAIN
```

The test scores are stored in a two-dimensional array (lines 5–9).

In lines 22–25, a column is summed and the total placed in the array AVG. In lines 28–30, this total is divided by 5 to compute the column average.

Rows and columns of array SCORE are indexed by BX and SI, respectively. We choose to begin summing column 4; this column begins in SCORES+6, so SI is initialized to 6 (line 16). After a column is summed, SI is decreased by 2, until it is 0.

The execution of the program may be seen in DEBUG. We execute down to the DOS exit, then dump the array AVG (the addresses in this demonstration were determined in a previous DEBUG session).

```
-G29
AX=4C4B BX=0028 CX=0000 DX=0002 SP=0100 BP=0000 SI=FFFE DI=0000
DS=100B ES=0FF9 SS=100F CS=1009 IP=0029 NV UP EI NG NZ AC PO CY
1009:0029 CD21    INT   21

-D36 3D
100B:0030      4B 00-3F 00 5C 00 2D 00
```

The averages are 004Bh, 003Fh, 005Ch, and 002Dh, or—in decimal 75, 63, 92, and 45.

10.7 The XLAT Instruction

In some applications, it is necessary to translate data from one form to another. For example, the IBM PC uses ASCII codes for characters, but IBM mainframes use EBCDIC (Extended Binary Coded Decimal Interchange Code). To translate a character string encoded in ASCII to EBCDIC, a program must replace the ASCII code of each character in the string with the corresponding EBCDIC code.

The instruction **XLAT** (translate) is a no-operand instruction that can be used to convert a byte value into another value that comes from a table. The byte to be converted must be in AL, and BX has the offset address of the conversion table. The instruction (1) adds the contents of AL to the address in BX to produce an address within the table, and (2) replaces the contents of AL by the value found at that address.

For example, suppose the contents of AL are in the range 0 to Fh and we want to replace it by the ASCII code of its hex equivalent; for example, 6h by 036h = "6", Bh by 042h = "B". The conversion table is

```
TABLE  DB 030h,031h,032h,033h,034h,035,036h,037h,038h,039h
       DB 041h,042h,043h,044h,045h,046h
```

For instance, to convert 0Ch to "C", we do the following:

```
MOV AL,0Ch          ;number to convert
LEA BX,TABLE        ;BX has table offset
XLAT                ;AL has 'C'
```

Here XLAT computes address TABLE + Ch = TABLE + 12, and replaces the contents of AL by the number stored there, namely 043h = "C".

In this example, if AL contained a value *not* in the range 0 to 15, XLAT would translate it to some garbage value.

Example: Coding and Decoding a Secret Message

The following program prompts the user to type a message, encodes it in unrecognizable form, prints the coded message, translates it back, and prints the translation.
Sample output:

```
ENTER A MESSAGE:,
GATHER YOUR FORCES AND ATTACK AT DAWN, (input)
ZXKBGM WULM HUMPGN XJO XKKXPD XK OXSJ, (encoded)
GATHER YOUR FORCES AND ATTACK AT DAWN, (translated)
```

Algorithm for Coding and Decoding a Secret Message

```
Print prompt
Read and encode message
Go to a new line
Print encoded message
Go to a new line
Translate and print message
```

Program Listing PGM10_5.ASM

```
0:  TITLE PGM 10_5: SECRET MESSAGE
1:. MODEL    SMALL
2:  .STACK   100H
3:  .DATA
4:  ;alphabet                        ABCDEFGHIJKLMNOPQRSTUVWXYZ
5:  CODE_KEY DB 65 DUP (' '),    'XQPOGHZBCADEIJUVFMNKLRSTWY'
6:           DB 37 DUP (' ')
7:  DECODE_KEY DB 65 DUP (' '),  'JHIKLQEFMNTURSDCBVWXOPYAZG'
8:           DB 37 DUP (' ')
9:  CODED    DB 80 DUP ('$')
10: PROMPT   DB 'ENTER A MESSAGE:',0DH,0AH,'$'
11: CRLF     DB     0DH,0AH,'$'
12: .CODE
13: MAIN     PROC
14:          MOV    AX,@DATA      ;initialize DS
15:          MOV    DS,AX
16: ;print input prompt
17:          MOV    AH,9          ;print string fcn
18:          LEA    DX,PROMPT     ;DX pts to prompt
19:          INT    21H           ;print message
20: ;read and encode message
21:          MOV    AH,1          ;read char fcn
22:          LEA    BX,CODE_KEY   ;BX pts to code key
23:          LEA    DI,CODED      ;DI pts to coded message
24: WHILE_:
25:          INT    21H           ;read a char
26:          CMP    AL,0DH        ;carriage return?
27:          JE     ENDWHILE      ;yes, go to print coded message
28:          XLAT                 ;no, encode char
```

```
29:           MOV    [DI],AL       ;store in coded message
30:           INC    DI            ;move pointer
31:           JMP    WHILE_        ;process next char
32:  ENDWHILE:
33:  ;go to a new line
34:           MOV    AH,9
35:           LEA    DX,CRLF
36:           INT    21H           ;new line
37:  ;print encoded message
38:           LEA    DX,CODED      ;DX pts to coded
39:           INT    21H           ;print coded message
40:  ;go to a new line
41:           LEA    DX,CRLF
42:           INT    21H           ;new line
43:  ;decode message and print it
44:           MOV    AH,2          ;print char fcn
45:           LEA    BX,DECODE_KEY ;BX pts to decode key
46:           LEA    SI,CODED      ;SI pts to encoded message
47:  WHILE1:
48:           MOV    AL,[SI]       ;get a character from message
49:           CMP    AL,'$'        ;end of message?
50:           JE     ENDWHILE1     ;yes, exit
51:           XLAT                 ;no, decode character
52:           MOV    DL,AL         ;put in DL
53:           INT    21H           ;print translated char
54:           INC    SI            ;move ptr
55:           JMP    WHILE1        ;process next char
56:  ENDWHILE1:
57:           MOV    AH,4CH
58:           INT    21H           ;dos exit
59:  MAIN     ENDP
60:           END    MAIN
```

Three arrays are declared in the data segment:

1. CODE_KEY is used to encode English text.
2. CODED holds the encoded message; it is initialized to a string of dollar signs so that it may be printed with INT 21h, function 9.
3. DECODE_KEY is used to translate the encoded text back to English.

Line 4 is a comment line containing the alphabet, which makes it easier to see how characters are encoded and decoded.

In lines 24–32, characters are read and encoded until a carriage return is typed. AL receives the ASCII code of each input character; XLAT adds it to address CODE_KEY in BX to produce an address within the CODE_KEY table.

CODE_KEY is set up as follows: 65 blanks, followed by the letters to which A to Z will be encoded, followed by 37 more blanks for a total of 128 bytes (128 bytes are needed, because the standard ASCII characters range from 0 to 127). Suppose, for example, an "A" is typed. The ASCII code of "A" is 65. XLAT computes address CODE_KEY+65, picks up the value of that byte, which is "X", and stores it in AL. At line 33, this value is moved into byte array CODED. Similarly, "B" is translated into 'Q', 'C' into 'P' . . . 'Z' into "Y" (the encoding table was constructed arbitrarily). Characters other than capital letters (including the blank character) have ASCII code in the

ranges 0 to 64 or 92 to 127, and are translated into blanks. In lines 38–39, the encoded message is printed.

DECODE_KEY also begins with 65 blanks and ends with 37 blanks. The positions of the letters in this array may be deduced as follows. First, lay down the alphabet (line 4). Now since "A" was coded into "X", the letter at position "X" in the decoding sequence should be "A". Similarly, because "B" was coded into "Q", there should be a "B" at position "Q", and so on.

In lines 47–56, the encoded message is translated. After placing the addresses of DECODE_KEY and CODED in BX and SI, respectively, the program moves a byte of the coded message into AL. If it's a dollar sign, the message has been translated and the program exits. If not, XLAT adds AL to address DECODE_KEY to produce an address within the decoding table, and puts the character found there into AL. At line 52, the character is moved to DL so that it can be printed with INT 21h, function 2.

Summary

- A one-dimensional array is an ordered list of elements of the same type. The DB and DW pseudo-ops are used to declare byte and word arrays.
- An array element can be located by adding a constant to the base address.
- The way that an operand is specified is its addressing mode. The addressing modes are register, immediate, direct, register indirect, based, indexed, and based indexed.
- In register indirect mode, an operand has the form [register], where register is BX, SI, DI, or BP. The operand's offset is contained in the register. For BP, the operand's segment number is in SS; for the other registers, the segment number is in DS.
- In based or indexed mode, an operand has the form [register + displacement]. Register is BX, BP, SI, or DI. The operand's offset is obtained by adding the displacement to the contents of the register. For BX,SI, or DI, the segment number is in DS; for BP, the segment number is in SS.
- The operators BYTE PTR and WORD PTR in front of an operand may be used to override the operand's declared type.
- The LABEL pseudo-op may be used to assign a type to a variable.
- A two-dimensional array is a one-dimensional array whose elements are one-dimensional arrays. Two-dimensional arrays may be stored row by row (row-major order), or column by column (column-major order).
- In based indexed mode, the offset address of the operand is the sum of (1) BX or BP; (2) SI or DI; (3) optionally, a memory offset address; (4) optionally, a constant. One (of several) possible forms is [base_register + index_register + memory_location + constant]. DS has the segment number if BX is used; if BP is used, SS has the segment number.
- Based indexed mode may be used to process two-dimensional arrays.
- The XLAT instruction can be used to convert a byte value into another value that comes from a table. AL contains the value to be

converted and BX the address of the table. The instruction adds AL to the offset contained in BX to produce a table address. The contents of AL is replaced by the value found at that address.

Glossary

addressing mode	The way the operand is specified
base address of an array	The address of the array variable
based addressing mode	An indirect addressing mode in which the contents of BX or BP are added to a displacement to form an operand's offset address
column-major order	Column by column
direct mode	The operand is a variable
displacement	In based or indexed mode, a number added to the contents of a register to produce an operand's offset address
immediate mode	The operand is constant
indexed addressing mode	An indirect addressing mode in which the contents of SI or DI are added to a displacement to form an operand's offset address
one-dimensional array	An ordered list of element of the same type
pointer	A register that contains an offset address of an operand
register mode	The operand is a register
row-major order	Row by row
two-dimensional array	A one-dimensional array whose elements are one-dimensional arrays

New instructions

XLAT

New Pseudo-Ops

DUP LABEL PTR

Exercises

1. Suppose

AX contains 0500h
BX contains 1000h
SI contains 1500h
DI contains 2000h

offset 1000h contains 0100h
offset 1500h contains 0150h
offset 2000h contains 0200h
offset 3000h contains 0400h
offset 4000h contains 0300h

and BETA is a word variable whose offset address is 1000h

For each of the following instructions, if it is legal, give the source offset address or register and the result stored in the destination.

a. `MOV DI,SI`

b. `MOV DI,[DI]`

c. `ADD AX,[SI]`

d. `SUB BX,[DI]`

e. `LEA BX,BETA[BX]`

f. `ADD [SI],[DI]`

g. `ADD BH,[BL]`

h. `ADD AH,[SI]`

i. `MOV AX,[BX + DI + BETA]`

2. Given the following declarations

```
A   DW   1,2,3
B   DB   4,5,6
C   LABEL  WORD
MSG DB   'ABC'
```

and suppose that BX contains the offset address of C. Tell which of the following instructions are legal. If so, give the number moved.

a. `MOV  AH, BYTE PTR A`

b. `MOV  AX, WORD PTR B`

c. `MOV  AX, C`

d. `MOV  AX, MSG`

e. `MOV  AH, BYTE PTR C`

3. Use BP and based mode to do the following stack operations. (You may use other registers as well, but don't use PUSH or POP.)
   a. Replace the contents of the top two words on the stack by zeros.
   b. Copy a stack of five words into a word array ST_ARR, so that ST_ARR contains the stack top, ST_ARR + 2 contains the next word on the stack, and so on.

4. Write instructions to carry out each of the following operations on a word array A of 10 elements or a byte array B of 15 elements.
   a. Move A[i+1] to position i, i = 1 . . . 9, and move A[1] to position 10.
   b. Count in DX the number of zero entries in array A.
   c. Suppose byte array B contains a character string. Search B for the first occurrence of the letter "E". If found, make SI point to its location; if not found, set CF.

5. Write a procedure FIND_IJ that returns the offset address of the element in row i and column j in a two-dimensional $M \times N$ word array A stored in row-major order. The procedure receives i in AX, j in BX, N in CX, and the offset of A in DX. It returns the offset address of the element in DX. *Note:* you may ignore the possibility of overflow.

Programming Exercises

6. To sort an array A of N elements by the bubblesort method, we proceed as follows:

   Pass 1. For $j = 2 \ldots N$, if A[j] < A[j – 1] then swap A[j] and A[j – 1]. This will place the largest element in position N.

   Pass 2. For $j = 2 \ldots N - 1$, if A[j] < A[j – 1] then swap A[j] and A[j – 1]. This will place the second largest element in position $N - 1$.

   .

   .

   .

   Pass N – 1. If A[2] < A[1], then swap A[2] and A[1]. At this point the array is sorted.

   Demonstration

initial data	7	5	3	9	1
pass 1	5	3	7	1	9
pass 2	3	5	1	7	9
pass 3	3	1	5	7	9
pass 4	1	3	5	7	9

   Write a procedure BUBBLE to sort a byte array by the bubblesort algorithm. The procedure receives the offset address of the array in SI and the number of elements in BX. Write a program that lets the user type a list of single-digit numbers, with one blank between numbers, calls BUBBLE to sort them, and prints the sorted list on the next line. For example,

```
?2 1 6 5 3 7
1 2 3 5 6 7
```

   Your program should be able to handle an array with only one element.

7. Suppose the class records in the example of section 10.4.3 are stored as follows

```
CLASS
DB      'MARY ALLEN  ',67,45,9 8,33
DB      'SCOTT BAYLIS',70,56,87,44
DB      'GEORGE FRANK',82,72,89,40
DB      'SAM WONG    ',78,76,92,60
```

   Each name occupies 12 bytes. Write a program to print the name of each student and his or her average (truncated to an integer) for the four exams.

8. Write a program that starts with an initially undefined byte array of maximum size 100, and lets the user insert single characters

into the array in such a way that the array is always sorted in ascending order. The program should print a question mark, let the user enter a character, and display the array with the new character inserted. Input ends when the user hits the ESC key. Duplicate characters should be ignored.

Sample execution:

```
?A
A
?D
AD
?B
ABD
?a
ABDa
?D
ABDa
? <ESC>
```

9. Write a program that uses XLAT to (a) read a line of text, and (b) print it on the next line with all small letters converted to capitals. The input line may contain any characters—small letters, capital, letters, digit characters, punctuation, and so on.
10. Write a procedure PRINTHEX that uses XLAT to display the content of BX as four hex digits. Test it in a program that lets the user type a four-digit hex integer, stores it in BX using the hex input algorithm of section 7.4, and calls PRINTHEX to print it on the next line.